rogii-wellbore-geology-prediction

17位解法

Rank	17
Team	Falcon
Author	Falcon
Topic ID	733860
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733860
Retrieved with Kaggle CLI on 2026-08-10.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

1. Solution overview

final submissionは3 modelのweighted averageです。Model 1 + Model 2 + Model 3で、**Private 6.376 / Public 5.853、総合17位**となりました。

```
Model 1  direct prediction     L      non-learned estimator上のGBDT。最終既知TVTからの相対TVTを予測
Model 2  integrated slope      Σr     CNNで予測したlocal slopeをlast_known_tvtから積分してTVTを得る
Model 3  error correction      δ      600 ft segmentごとに推定したsystematic error

pred = (1−w)·(last_known_tvt + L)  +  w·(last_known_tvt + Σr)  −  λ·δ
       └──── Model 1 ────┘      └── Model 2 ──┘        └ Model 3 ┘

w = 0.40    weighted averageのweight
λ(<1200 / 1200–3000 / ≥3000 ft) = (1.25, 1.45, 1.60)    correction strength
```

#	Configuration	CV	Public LB	Private LB
0	GR-likelihood particle filterのみ	11.08	—	—
1	Model 1	7.09	6.311	7.827
2	Model 1 + 2	6.66	6.130	6.912
3	Model 1 + 2 + 3	**6.32**	**5.853**	6.376

Local validation

training dataをwell単位でGroupKFold（K=5）に分けてvalidationしました。ただしwellごとに難易度が大きく異なるため、RMSEだけで変更を採用するか決めるのは不十分だと考えました。そこで最もnaiveなestimatorであるGR-likelihood particle filterのwell別RMSEを使い、wellを4つのdifficulty levelへ分け、level別RMSEも評価しました。

Level	Definition（PFのper-well RMSE）	Wells
normal	< 5 ft	332
mid	5–10 ft	258
high	10–20 ft	133
catastrophic	≥ 20 ft	50

なぜこれが必要だったか

RMSEはquadratic meanなので、wellの6%しかないcatastrophic levelがCV scoreを大きく動かします。このwell群だけを改善する変更は、majorityのwellを全く改善しなくてもCV上は良く見えます。そして実際、「hard wellを直し、normal levelを傷つける」類の変更は、ほとんどがPublic LBを悪化させました。

そのため、experimentのsuccess criterionをoverall RMSEだけでなく、「全levelで改善し、特にnormal levelを大きく改善すること」としました。AI agentへ自律的にmethod探索をさせながら、安定したimprovement cycleを回すうえで、これが不可欠でした。

Model 1

基本的にはpublic notebookで公開されたapproachを土台にしています。targetは`TVT − last_known_tvt`で、modelは`CatBoost:LGBM:XGB = 2:1:1`です。

264 featuresを使い、その中心はnon-learned estimator群です。

- single well（自身のGRとtypewell）だけで動くestimator：GR-likelihood PF、beam search、multi-scale NCCなど
- neighbouring training wellsからlabelを借りるestimator：近傍wellのlabeled TVT-GRから作った「pseudo typewell」に対するPF予測、neighboring structural surface、isochore、dip、coverageなど

Model 2

Model 1には、使うestimatorに依存して予測全体がほぼ一定量shiftするdriftがありました。そこで、このdriftを受けにくいmodelを作り、Model 1とweighted averageしました。

- Target：TVT自体ではなくlocal slope。予測slopeを`last_known_tvt`から再積分してTVT predictionへ戻す
- Input：1 well = 1 sequence（1 ft step）、74 features。既存estimatorをslopeへ変換したもの、trajectory ZのvelocityとcurvatureとGRとそのstatistics、estimator間disagreementなど

Architecture：

```
input   [74ch × 1024ft]
  ↓  1D Conv ×3 (kernel 32 / 16 / 4, stride 2, 64ch each) → [3ch × 64 × 512]
  ↓  EfficientNet-B0
  ↓  frequency axisをaverageし、1D sequenceへ戻す
  ↓  1D Conv head (256 → 128 → 1) → input lengthへlinear interpolation
output  [1024ft全体のslope]
```

Model 3

Model 1と2をweighted averageした後にも、segment全体で予測が上下へshiftするerrorが残りました。そこでsegmentごとにそのshiftをregressionし、予測から引くmodelを作りました。

- Target：600 ft segmentごとの予測shift（segment内mean residual）
- Model：LightGBM
- Input：160 features。主に各estimator predictionとModel 1+2 predictionとの差、およびestimator間のspread

AI agentと「errorを分析し、改善を設計する」ことを繰り返した結果、residual errorの上にcorrection modelをstackするのが最も実行しやすく、最終的にこの構造となりました。

2. AI agentとの開発

coding agent（Claude Code、Opus 4.8 / Opus 5 / Fable）を使い、約2か月で次の規模の探索を行いました。

項目	Count
Experiment scripts	136
Feature-generation scripts	52
agentが実行したanalysis / measurement scripts	750+
Agent memories（1 finding per file）	232

2.1 役割分担

以下の分担で進め、私自身の役割は探索方法を決め、各methodを採用するか判断することでした。自分ではほとんどcodeを書いていません。

作業	Owner
problem分析、idea提案	Agent
problem formulation、探索方向の選択、hypothesis提案	Human
implementation（feature block、training script、measurement harness）と実行	Agent
result解釈、採用判断、final submission選択	Human
experiment ledgerとfindingの維持、次sessionへの引き継ぎ	Agent

2.2 探索方向の定義

agentはmeasurementとimplementationに強い一方、結果が少し出るとsearch spaceを早く閉じる傾向があります。数個のexperiment後に「この方向は完全に探索された」と結論し、次のproposalへ移ろうとします。そのanalysisとproposalを額面どおり受け取ると、誤った結論から探索を続け、最終的に行き詰まります。そのためhumanが方向性を理解し、妥当か判断することが重要でした。

例えばresidualの主成分が「wellごとに一定量shiftするerror」だと分かった直後、agentは「GR、well-level feature、local geometryはいずれもこのshiftについて情報を持たない」と結論し、その方向を閉じようとしました。しかし実際に棄却されたのは、naive GR matchingとaggregateした既存feature上のregressionという2つの具体的estimatorだけであり、「shiftを予測する」という方向全体ではありませんでした。反論して探索を続けたことがModel 3につながりました。いったん閉じられた方向が、最終的には実際の改善を生みました。

同様に、correctionがwellごとの1 constantという最もnaiveな形に留まっていた時期がありました。十分に探索されていないと指摘したことで、600 ft segmentごとにcorrectionする形へ進み、scoreがさらに改善しました。実際に効いたmethodとfeatureは、探索を早く閉じさせず続けたことで得られました。

2.3 AI agentで難しかったこと／うまくいかなかったこと

- 蓄積memoryがagentの思考を固定し、investigation directionがrigidになる

- LLMはtimeとcostの概念を持ち、cheap methodだけを試そうとする傾向がある。コンペ終盤にはsimple approachばかり提案するようになる
- 生成されたanalysisとimplementationが複雑すぎて追えなくなり、humanが意味のないexperimentを依頼してしまう

2.4 Summary

implementationとmeasurementをagentへ任せることでiteration数は大幅に増えました。

- autonomyを高めるには、何をimprovementと数えるかのcriterionを固定することが重要
- agentのanalysisとproposalを額面どおり受け取らず、humanがdirectionを判断して返す必要がある
- Kaggle competitionほど複雑なtaskをLLMで解くには、investigationを簡単に閉じないよう指示するなどのtuningが必要
- compute resourceを考慮し、cheap methodやmarginal gainだけへ集中しないよう指示するなど、探索のexpected valueが最大になる方向へLLMを導くことが重要